Bridging Evolution and Learning: Attention-Guided Co-Design of Modular Soft Robots

Yuchao Du
Jilin University
duyc24@mails.jlu.edu.cn

Meibao Yao*
Jilin University
meibaoyao@jlu.edu.cn

***Abstract*—In the co-design of morphology and control for modular soft robots, the diverse combinations of module types and spatial layouts cause the morphology search space to expand rapidly, while evaluating candidate morphologies requires computationally expensive controller training. To improve co-design efficiency under a limited training budget, we propose Attention-Guided Co-Design (AGCD). The framework uses controller attention as a link, applying information acquired during control policy learning to guide morphological crossover and mutation, thereby connecting learning and evolution. It also allocates training resources by combining staged evaluation with Training Level–Fitness Non-Dominated Sorting, preserving opportunities for further optimization of undertrained but promising candidate morphologies while balancing morphology exploration and controller optimization. Experiments on six Evolution Gym tasks under the same total training budget show that AGCD explores more candidate morphologies and achieves higher final mean rewards than the baseline methods on all tasks.**

***Index Terms*—Co-design, Modular Soft Robots, Evolutionary Algorithm**

I. Introduction

Robot morphology and control are tightly coupled: a change in body structure alters the dynamics that a controller must learn, and the apparent quality of a body depends on how thoroughly its controller has been trained. This interaction is fundamental to embodied intelligence [1], [2] and has motivated automated searches over bodies and behaviors since early evolutionary robotics studies [3]–[5]. The challenge is especially acute for voxel-based soft robots. Even a small grid admits a large number of material layouts, while evaluating each layout requires costly controller training [6], [7]. Allocating the full preset controller-training budget to every candidate can therefore exhaust the search budget before enough distinct morphologies are explored.

Efficient co-design under a fixed budget requires decisions about both evaluation and variation: which candidates should receive more controller training, and how should existing candidates inform the next generation? Population-based methods explore multiple morphologies in parallel [8]–[10], but selecting among them remains difficult when fitness estimates depend on training effort. Moreover, a structural change can diminish the usefulness of a previously trained controller [11]. Reinforcement-learning-based approaches provide another route to joint design [12]–[15], but they still require training resources to be allocated across candidate structures. An effective co-design method should therefore manage incomplete evaluations and exploit information gained during controller training to guide morphology search.

To address these challenges, we propose Attention-Guided Co-Design (AGCD), which converts attention information formed during control policy learning into a guidance signal for morphology evolution. Reinforcement learning gradually adapts the controller to the current morphology, while self-attention integrates information from individual voxels when generating control actions. AGCD aggregates the attention weights of the trained controller along sampled trajectories into voxel-level scores to guide crossover and mutation: voxels with higher scores are preferentially retained, while other regions are modified to generate new module combinations. Information acquired during controller learning is thus fed back into the production of the next generation of morphologies, connecting learning and evolution during co-design.

To balance morphology exploration and controller optimization under a limited budget, AGCD evaluates candidates in stages and uses Training Level–Fitness Non-Dominated Sorting to rank them by observed fitness and training level, where training level measures cumulative controller-training effort. This ranking retains promising candidates that have received relatively little training, while a subsequent elite phase completes the preset per-candidate training budget for selected archived candidates. Our main contributions are:

- We propose AGCD, a co-design framework that connects control policy learning and morphology evolution through controller attention, enabling synergy between evolution and learning.
- We develop attention-guided variation operators that preferentially retain highly attended voxels while modifying other regions, incorporating controller-derived attention information into offspring generation.
- We introduce a training-budget allocation strategy that combines staged evaluation with Training Level–Fitness Non-Dominated Sorting to retain undertrained but promising candidates.

II. Related Work

A. Robot Co-Design

Robot co-design aims to jointly optimize morphology and control policies. Because morphology and controllers are

*Corresponding author: Meibao Yao.

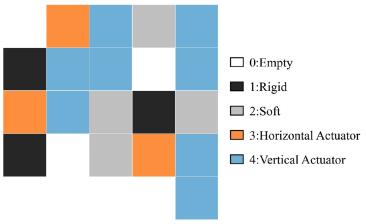
closely interdependent [16], [17], morphology design often directly affects the expressive capacity of control policies. Previous studies have examined how different encoding approaches affect co-optimization performance [6], [18], [19]. These approaches are generally categorized as direct or indirect encoding. Among indirect encoding approaches, the compositional pattern-producing network (CPPN) [19] is an efficient method for generating morphologies with symmetry and repeated structures. In contrast, direct encoding often represents robot morphologies as graphs [13], effectively modeling relationships between modules. Beyond single-task co-design, this paradigm has been extended to multi-task settings [20], facilitating knowledge transfer from simple to more complex tasks [21], [22]. Generative models have also been introduced into co-design, including classifier-guided diffusion models for improving the efficiency of morphology evolution [23], [24]. Large language models have also been explored as evolutionary search operators to balance optimization efficiency and solution diversity [25], [26].

B. Self-Attention

Self-attention models dependencies between positions through adaptively weighted aggregation of information within the same input sequence and is the core mechanism of the Transformer [27]. Its attention weights are $A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$, where Q, K, and V are the query, key, and value matrices obtained by linearly projecting the input, respectively, and d_k is the dimension of the key vectors. The attention weights A are then used to compute weighted sums of the vectors in the value matrix V, integrating information from different positions. In voxel-based soft robots, self-attention enables the controller to integrate module observations from different positions, supporting global information exchange and action coordination. Wang et al. [12] use this mechanism to construct control policies for voxel-based robots. We further use attention information from the controller to guide morphology variation operators, allowing information acquired during control policy learning to participate in morphology evolution.

III. Problem Formulation

The search studied here assigns a material to each cell of a two-dimensional design grid, then evaluates the resulting robot after controller training. This grid representation makes structural changes discrete, but each change can also alter the behavior that a controller must learn. Voxelized designs have been used to study automated soft-robot design and morphological diversity [29], [30]. We use the Evolution Gym representation [6]: the five cell values are empty (0), rigid material (1), soft material (2), horizontal actuator (3), and vertical actuator (4), as illustrated in Fig. 1. The two actuator types change length along their respective axes; soft material deforms passively, while rigid material retains its shape. Adjacent non-empty voxels transmit forces, allowing the assembled body to move. Let the morphology be selected from a discrete set and its controller from a parameterized policy family. The idealized task objective comprises an outer



(a)

0	3	4	2	4
1	4	4	0	4
3	4	2	1	2
1	0	2	3	4
0	0	0	0	4

(b)

Fig. 1. Example of a voxel-based soft robot (VSR). (a) Robot structure. (b) Corresponding numerical encoding.

morphology selection and an inner controller selection for that morphology:

$$\begin{aligned}(s^*, \phi_s^*) &= \arg\max_{s \in \mathcal{S}} \; F(s, \phi_s^*)\,, \\ \text{s.t. } \phi_s^* &= \arg\max_{\phi \in \Theta} \; f(s, \phi)\,.\end{aligned} \tag{1}$$

Here, $s \in \mathcal{S}$ specifies the grid of voxel types, and $\phi \in \Theta$ is the parameter vector of its neural controller. Without connectivity and actuator constraints, an $n \times n$ grid has 5^{n^2} possible assignments; only a subset forms valid robots. The inner objective $f(s, \phi)$ describes controller performance for a fixed body, while the outer objective $F(s, \phi_s^*)$ scores a body equipped with its optimized controller. In practice, reinforcement learning (RL) only approximates this inner optimum using a finite number of simulation steps. Fitness therefore changes as controller training proceeds. AGCD allocates this limited training budget across candidate morphologies rather than requiring completion of the preset per-candidate training budget before every structural selection decision.

IV. Proposed Method

AGCD comprises two phases, Staged Evolution and Elite Training, as shown in Fig. 2, with the overall procedure given in Algorithm 1. During Staged Evolution, candidates obtain fitness estimates through incremental controller training. Training Level–Fitness Non-Dominated Sorting selects survivors, and attention-guided variation operators generate offspring. Elite Training subsequently completes the remaining controller training for selected archived individuals and determines the final robot design based on task performance.

A. Staged Evaluation and Non-Dominated Sorting

The observed fitness of a candidate morphology depends on the training its controller has received. To explore more morphologies under a limited budget, AGCD divides controller training into fixed-length intervals and defines training level as the number of completed intervals. Each evaluation adds one interval of training to a candidate, updates its fitness estimate, and increases its training level by one. The training level limit is 25, corresponding to the preset per-candidate training budget.

To compare candidates with different amounts of training, AGCD uses Training Level–Fitness Non-Dominated Sorting,

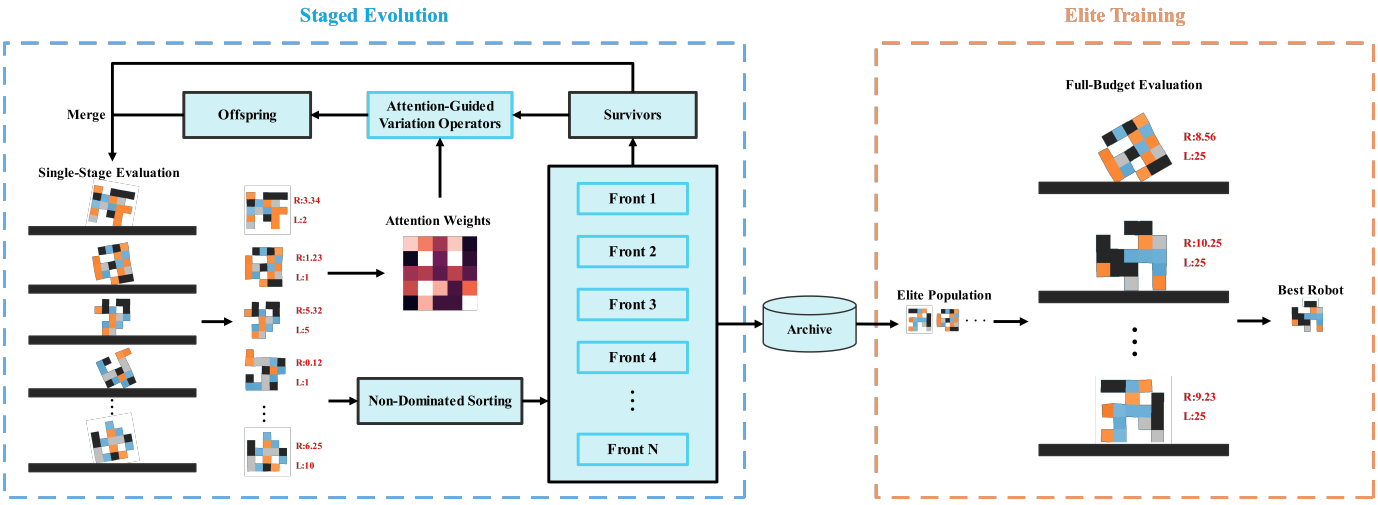


Fig. 2. The AGCD framework. Left: candidates are trained and evaluated in stages, ranked by fitness and training level using non-dominated sorting, and used to generate offspring guided by controller attention. Right: elite individuals are selected from the archive, their remaining training is completed, and the best robot is output. R and L denote reward and training level, respectively.

Algorithm 1. Pseudocode of AGCD

Input: N: population size; $L_{\max}$: level limit; T_l: iterations per level; $T_{\max}$: total budget; T_e: elite reserve.
Output: (s^*, ϕ_s^*).

$P \leftarrow \text{Initialize}(N)$; $U \leftarrow P$; $\mathcal{A} \leftarrow \varnothing$; $T_{\text{cur}} \leftarrow 0$.
while $T_{\text{cur}} + |U|T_l + T_e \le T_{\max}$ **do**
  for each $p \in U$ **do**
    $p \leftarrow \text{TrainEval}(p, T_l)$; $\mathbf{a}_p \leftarrow \text{AttentionScore}(p)$.
    $p_l \leftarrow p_l + 1$; $T_{\text{cur}} \leftarrow T_{\text{cur}} + T_l$; $\mathcal{A} \leftarrow \text{Update}(\mathcal{A}, p)$.
  end for
  $S \leftarrow \text{Select}(P, N/2)$, Eq. (2).
  $C_1 \leftarrow \text{Crossover}(S, N/4)$, Eq. (3).
  $C_2 \leftarrow \text{Mutation}(S, N/4)$, Eq. (4).
  $P \leftarrow S \cup C_1 \cup C_2$; $U \leftarrow \text{Active}(P, L_{\max})$.
end while
$E \leftarrow \text{SelectElites}(\mathcal{A}, T_e)$.
for each $e \in E$ **do**
  $r_e \leftarrow (L_{\max} - e_l)T_l$; $e \leftarrow \text{TrainEval}(e, r_e)$.
  $e_l \leftarrow L_{\max}$; $T_{\text{cur}} \leftarrow T_{\text{cur}} + r_e$; $\mathcal{A} \leftarrow \text{Update}(\mathcal{A}, e)$.
end for
$p^* \leftarrow \arg\max_{p \in \mathcal{A}} p_f$; $(s^*, \phi_s^*) \leftarrow (s_{p^*}, \phi_{p^*})$.
return (s^*, ϕ_s^*).

considering both higher fitness and lower training levels. This method partitions candidates based on Pareto dominance [31], [32]. Let $a \succ b$ denote that candidate a dominates candidate b. The dominance criterion is:

$$a \succ b \iff \begin{cases} a_l \le b_l, \\ a_f \ge b_f, \\ (a_l < b_l) \vee (a_f > b_f) \end{cases} \tag{2}$$

where the subscripts l and f denote training level and observed fitness, respectively. Thus, a has received no more training than b, has fitness no lower than that of b, and is strictly better in at least one of these two respects. This selection method considers both task performance and training effort, preserving opportunities for further optimization of promising candidate morphologies that have received less training.

We use the fast non-dominated sorting procedure of NSGA-II [32], preferentially retaining individuals on earlier non-dominated fronts and selecting individuals within the same front in descending order of fitness. After Staged Evolution, elite individuals are selected from the archive and allocated their remaining training budgets until they reach the training level limit.

B. Attention-Guided Variation Operators

AGCD uses controller attention information to guide morphology changes, with variation operators comprising crossover and mutation. After evaluating a candidate, we sample a simulation trajectory using its controller and record the attention matrix at each time step. We first average these matrices over time, then sum each column of the averaged attention matrix to aggregate the attention assigned to each voxel by all query positions. Finally, we normalize the result to obtain voxel-level scores. The two operations use these scores to determine which voxels to retain or modify, applying information acquired during control policy learning to offspring generation.

1) Attention-Guided Crossover Operation: The crossover operation determines the offspring's material layout at each grid position based on the voxel types and attention scores of two parents. Let v_i^c and a_i^c denote the encoded voxel type and attention score, respectively, of parent P_i ($i \in \{1, 2\}$) at

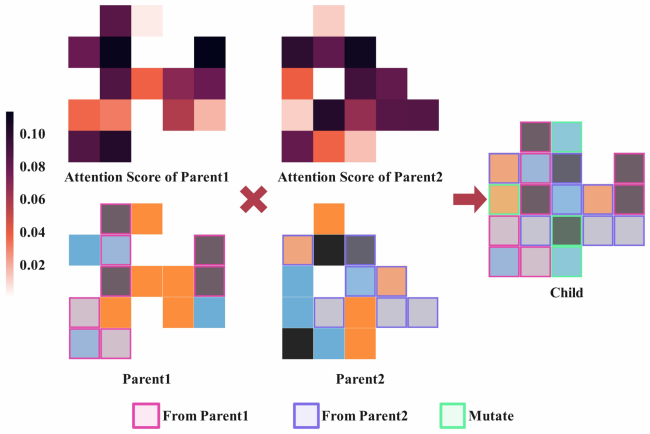


Fig. 3. Example of the attention-guided crossover operation.

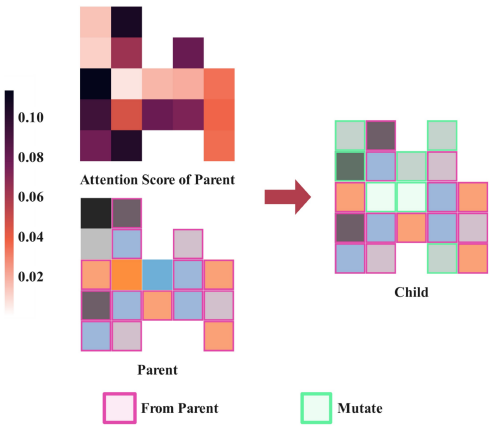


Fig. 4. Example of the attention-guided mutation operation.

position c. The offspring voxel is assigned according to:

$$v_{off}^{c} = \begin{cases} v^{c}_{\underset{i\in\{1,2\}}{\arg\max}\, a_i^c} & \text{if } v_1^c \neq 0,\ v_2^c \neq 0 \\ v_i^c & \text{if } v_i^c \neq 0,\ v_{3-i}^c = 0,\ a_i^c \geq \tilde{a}_i \\ \text{mutate}(c) & \text{if } v_i^c \neq 0,\ v_{3-i}^c = 0,\ a_i^c < \tilde{a}_i \\ 0 & \text{if } v_1^c = v_2^c = 0 \end{cases} \tag{3}$$

where $\tilde{a}_i$ is the median score of all non-empty voxels in parent P_i, and $\text{mutate}(c)$ randomly replaces the voxel type at position c with another type. When both parents have a non-empty voxel at the same position, the voxel from the parent with the higher score is inherited. When only one parent has a non-empty voxel, its score determines whether it is retained or mutated. When both parents have an empty voxel, the position remains empty. In Fig. 3, pink and purple borders indicate voxels inherited from the two parents, respectively, while green borders indicate voxels produced by random mutation.

2) Attention-Guided Mutation Operation: The mutation operation takes a single parent and its voxel scores as input. It retains non-empty voxels whose scores are at least the median and applies random mutation to all remaining positions. Let v^c and a^c denote the encoded voxel type and attention score, respectively, of parent P at position c. The offspring voxel is updated according to:

$$v_{off}^{c} = \begin{cases} v^c & \text{if } v^c \neq 0 \text{ and } a^c \geq \tilde{a} \\ \text{mutate}(c) & \text{if } v^c \neq 0 \text{ and } a^c < \tilde{a} \\ \text{mutate}(c) & \text{if } v^c = 0 \end{cases} \tag{4}$$

where $\tilde{a}$ is the median score of all non-empty voxels in parent P, and $\text{mutate}(c)$ is defined as in Eq. (3). This operation retains high-scoring voxels while changing the assignments of low-scoring voxels and empty positions to explore new material combinations and spatial layouts, as shown in Fig. 4.

After generating offspring, both operations check structural connectivity and the presence of actuator voxels. If these constraints are not satisfied, offspring are regenerated until a valid offspring is obtained or a preset maximum number of attempts is reached. Only offspring that pass these checks are accepted.

C. Control and Training Strategies

AGCD adopts the self-attention-based control policy proposed by Wang et al. [12]. It encodes each voxel's local observations and material information as a sequence, models dependencies between voxels through self-attention, and generates coordinated control actions for actuator voxels. The controller is trained using Proximal Policy Optimization (PPO) [33], which updates the policy parameters using task rewards as the learning signal. The attention weights produced during trajectory sampling are converted into voxel scores as described in Section IV-B to guide subsequent morphology evolution.

V. Experiments

We evaluate AGCD through comparative experiments and ablation studies to answer two questions: (1) Can attention effectively guide morphology search in co-design? (2) How do attention-guided variation and training-level-based allocation each contribute to co-design performance?

A. Experimental Settings

All experiments are implemented in Evolution Gym [6]. We set the robot design space to 5×5, define the maximum number of training iterations per individual for each task as t, and set the total number of iterations to $100t$ for easy tasks, $200t$ for medium tasks, and $300t$ for hard tasks. We evaluate AGCD on six task environments: two easy tasks (Walker-v0 and Jumper-v0), two medium tasks (Climber-v0 and Climber-v1), and two hard tasks (Climber-v2 and Hurdler-v0). To better compare AGCD with other methods, we use the following three categories of baseline methods:

- **GA:** The genetic algorithm is a classical evolutionary computation method that simulates natural selection, inheritance, and mutation to select and vary candidate solutions, iteratively generating better individuals.

- **BO:** Bayesian optimization is a surrogate-assisted algorithm for solving expensive black-box optimization problems [34]. It typically uses Gaussian processes to approximate the true objective function, thereby reducing the computational overhead of expensive fitness evaluations.
- **AIEA:** The Act_Inh-based evolutionary algorithm is a fast fitness evaluation algorithm [7]. Candidate robots inherit actions generated by well-trained controllers and combine them with error estimation to obtain approximate task performance.

AGCD adopts a $(\mu+\lambda)$ evolutionary strategy [35], where $\mu = 10$ and $\lambda = 10$. To maintain consistency across experiments, the population size of all other methods is set to 20, and the PPO parameter settings are identical. Detailed experimental parameters and the code implementation are available in the project repository at https://github.com/LittleDu26/AGCD.

B. Results and Analysis

Fig. 5 shows the reward curves of AGCD, AIEA, GA, and BO on all six evaluated tasks.

Fig. 5 shows that AGCD achieves higher final mean rewards than the baseline methods on all six tasks. The final performance of AGCD and AIEA is relatively close on the easy tasks, whereas the gaps are larger on some climbing and hurdle-crossing tasks. For example, on Hurdler-v0, AGCD achieves a final mean reward of approximately 3.0, compared with approximately 1.5 for AIEA, the best-performing baseline. Moreover, AGCD continues to improve after reward growth slows for some baselines, indicating that it can obtain better robot designs under the same training budget.

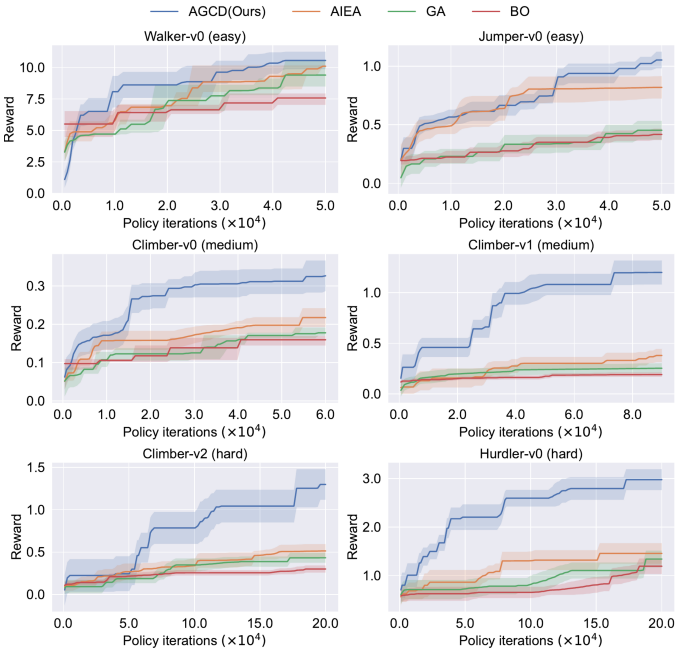


Fig. 5. Mean and standard deviation of the reward curves for AGCD, AIEA, GA, and BO on the six evaluated tasks. Each curve is averaged over five independent runs.

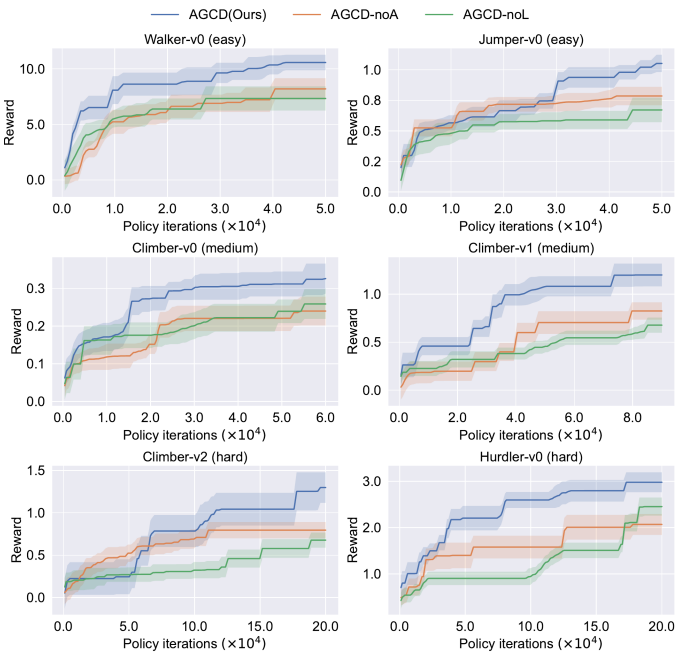


Fig. 6. Mean and standard deviation of the reward curves for AGCD, AGCD-noA, and AGCD-noL on the six evaluated tasks. Each curve is averaged over five independent runs.

C. Ablation Study

To validate the importance of each component of AGCD, we construct two variants while keeping all other parameters unchanged. The two variants are defined as follows:

- **AGCD-noA:** The attention-guided variation operators in AGCD are removed and replaced with the default variation operators used by the other baseline algorithms.
- **AGCD-noL:** Staged evaluation and Training Level–Fitness Non-Dominated Sorting are removed from AGCD, and each individual is evaluated using the full training budget.

Fig. 6 shows that removing either component reduces the final mean reward on all six tasks, supporting the effectiveness of the attention-guided variation operators and training-level-based budget allocation. Although some ablation variants are close to AGCD or briefly outperform it early in the search, the full framework achieves greater improvements later in the search. The relative performance of the two variants varies across tasks, indicating that the benefits of the two mechanisms differ by task, while combining them yields better final search results.

VI. Conclusion

We propose AGCD, an attention-guided co-design framework for modular soft robots that uses attention information formed during controller learning to guide morphological variation, allowing control policy learning to participate directly

in morphology evolution. Through staged evaluation, Training Level–Fitness Non-Dominated Sorting, and elite training, AGCD balances candidate morphology exploration and controller optimization under a limited budget. Under the same total training budget, AGCD achieves higher final mean rewards than the baseline methods on all six Evolution Gym tasks. The ablation experiments further support the effectiveness of the attention-guided variation operators and training-budget allocation strategy.

REFERENCES

[1] H. Liu, D. Guo, and A. Cangelosi, "Embodied intelligence: A synergy of morphology, action, perception and learning," *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.

[2] R. Pfeifer, F. Iida, and G. Gómez, "Morphological computation for adaptive behavior and cognition," in *International Congress Series*, vol. 1291. Elsevier, 2006, pp. 22–29.

[3] K. Sims, "Evolving virtual creatures," in *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '94. New York, NY, USA: Association for Computing Machinery, 1994, p. 15–22. [Online]. Available: https://doi.org/10.1145/192161.192167

[4] H. Lipson and J. B. Pollack, "Automatic design and manufacture of robotic lifeforms," *Nature*, vol. 406, no. 6799, pp. 974–978, 2000.

[5] J. C. Bongard and R. Pfeifer, "Evolving complete agents using artificial ontogeny," in *Morpho-functional machines: the new species: designing embodied intelligence*. Springer, 2003, pp. 237–258.

[6] J. Bhatia, H. Jackson, Y. Tian, J. Xu, and W. Matusik, "Evolution gym: A large-scale benchmark for evolving soft robots," *Advances in Neural Information Processing Systems*, vol. 34, pp. 2201–2214, 2021.

[7] S. Liu, W. Yao, H. Wang, W. Peng, and Y. Yang, "Rapidly evolving soft robots via action inheritance," *IEEE Transactions on Evolutionary Computation*, 2023.

[8] K. O. Stanley, D. B. D'Ambrosio, and J. Gauci, "A hypercube-based encoding for evolving large-scale neural networks," *Artificial life*, vol. 15, no. 2, pp. 185–212, 2009.

[9] F. Tanaka and C. Aranha, "Co-evolving morphology and control of soft robots using a single genome," in *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2022, pp. 1235–1242.

[10] K. Harada and H. Iba, "Lamarckian co-design of soft robots via transfer learning," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2024, pp. 832–840.

[11] N. Cheney, J. Bongard, V. SunSpiral, and H. Lipson, "Scalable co-optimization of morphology and control in embodied machines," *Journal of The Royal Society Interface*, vol. 15, no. 143, p. 20170937, 2018.

[12] Y. Wang, S. Wu, H. Fu, Q. Fu, T. Zhang, Y. Chang, and X. Wang, "Curriculum-based co-design of morphology and control of voxel-based soft robots," in *The Eleventh International Conference on Learning Representations*, 2023.

[13] T. Wang, Y. Zhou, S. Fidler, and J. Ba, "Neural graph evolution: Towards efficient automatic robot design," *arXiv preprint arXiv:1906.05370*, 2019.

[14] H. Lu, Z. Wu, J. Xing, J. Li, R. Li, Z. Li, and Y. Shi, "Bodygen: Advancing towards efficient embodiment co-design," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=cTR17xl89h

[15] J. Xu, T. Chen, L. Zlokapa, M. Foshey, W. Matusik, S. Sueda, and P. Agrawal, "An end-to-end differentiable framework for contact-aware robot design," *arXiv preprint arXiv:2107.07501*, 2021.

[16] A. Rosendo, M. Von Atzigen, and F. Iida, "The trade-off between morphology and control in the co-optimized design of robots," *PloS one*, vol. 12, no. 10, p. e0186107, 2017.

[17] H. Ma, Y. Zhang, S. Sun, T. Liu, and Y. Shan, "A comprehensive survey on nsga-ii for multi-objective optimization and applications," *Artificial Intelligence Review*, vol. 56, no. 12, pp. 15 217–15 270, 2023.

[18] F. Pigozzi, F. J. Camerota Verdù, and E. Medvet, "How the morphology encoding influences the learning ability in body-brain co-optimization," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2023, pp. 1045–1054.

[19] K. O. Stanley, "Compositional pattern producing networks: A novel abstraction of development," *Genetic programming and evolvable machines*, vol. 8, no. 2, pp. 131–162, 2007.

[20] A. Mertan and N. Cheney, "Towards multi-morphology controllers with diversity and knowledge distillation," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2024, pp. 367–376.

[21] J. Zhao, W. Peng, H. Wang, W. Zhou, Y. Yang, and W. Yao, "Cross-task collaborative optimization based on knowledge transfer for soft robot design," *IEEE Transactions on Evolutionary Computation*, 2025.

[22] Y. Wang, S. Wu, T. Zhang, Y. Chang, H. Fu, Q. Fu, and X. Wang, "Preco: Enhancing generalization in co-design of modular soft robots via brain-body pre-training," in *Conference on Robot Learning*. PMLR, 2023, pp. 478–498.

[23] T.-H. J. Wang, J. Zheng, P. Ma, Y. Du, B. Kim, A. Spielberg, J. Tenenbaum, C. Gan, and D. Rus, "Diffusebot: Breeding soft robots with physics-augmented generative diffusion models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 398–44 423, 2023.

[24] S. Liu, J. Yan, H. Wang, and Y. Jin, "Morphology evolution for embodied robot design with a classifier-guided diffusion model," *IEEE Transactions on Evolutionary Computation*, 2025.

[25] J. Song, Y. Yang, H. Xiao, W. Peng, W. Yao, and F. Wang, "Laser: Towards diversified and generalizable robot design with large language models," in *The Thirteenth International Conference on Learning Representations*, 2025.

[26] J. Fang, Y. Sun, C. Ma, Q. Lu, and L. Yao, "Robomore: Llm-based robot co-design via joint optimization of morphology and reward," *arXiv preprint arXiv:2506.00276*, 2025.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[28] J. Zhao, W. Peng, H. Wang, and W. Yao, "A multi-objective optimization framework based on information sharing for serially connected robot design," *Complex & Intelligent Systems*, vol. 11, p. 410, 2025.

[29] J. Hiller and H. Lipson, "Automatic design and manufacture of soft robots," *IEEE Transactions on Robotics*, vol. 28, no. 2, pp. 457–466, 2012.

[30] N. Cheney, R. MacCurdy, J. Clune, and H. Lipson, "Unshackling evolution: evolving soft robots with multiple materials and a powerful generative encoding," *SIGEVOlution*, vol. 7, no. 1, p. 11–23, Aug. 2014. [Online]. Available: https://doi.org/10.1145/2661735.2661737

[31] N. Srinivas and K. Deb, "Muiltiobjective optimization using nondominated sorting in genetic algorithms," *Evolutionary computation*, vol. 2, no. 3, pp. 221–248, 1994.

[32] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE transactions on evolutionary computation*, vol. 6, no. 2, pp. 182–197, 2002.

[33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[34] X. Wang, Y. Jin, S. Schmitt, and M. Olhofer, "Recent advances in bayesian optimization," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–36, 2023.

[35] H.-G. Beyer and H.-P. Schwefel, "Evolution strategies–a comprehensive introduction," *Natural computing*, vol. 1, no. 1, pp. 3–52, 2002.